

GUÍA INTEGRADA: APIs REST, VERBOS HTTP Y JSON

Propósito: Comprender cómo se comunican los sistemas modernos mediante APIs y cómo se intercambian datos usando JSON.

1. ¿Qué es una API?

API significa **Application Programming Interface** (Interfaz de Programación de Aplicaciones).

Una API es un **punto de comunicación entre sistemas** que permite que una aplicación solicite información o ejecute acciones en otra sin acceder directamente a su base de datos o lógica interna.

Ejemplo cotidiano

Una aplicación móvil solicita los datos de un usuario:

- La app envía una petición a la API.
- La API valida la solicitud.
- La API consulta la base de datos.
- La API responde con la información.

La API controla **qué se puede hacer, cómo se hace y quién puede hacerlo**.

2. ¿Cómo funciona una API?

Las APIs funcionan mediante el protocolo HTTP, usando:

- **Una URL** → el recurso al que quiero acceder.
- **Un verbo HTTP** → la acción que quiero realizar.
- **Un formato de datos (JSON)** → cómo viaja la información.

Ejemplo de solicitud:

```
GET /usuarios/15
```

Significa:

“Quiero consultar el usuario con ID 15.”

3. ¿Qué es un Recurso?

En una API todo gira alrededor de **recursos**, que representan entidades del sistema:

- usuarios

- productos
- pedidos
- facturas

Las URLs deben representar **cosas (sustantivos)**, no acciones.

Incorrecto:

```
/obtenerUsuarios
```

Correcto:

```
/usuarios
```

La acción la define el verbo HTTP.

4. Verbos HTTP (Las Acciones de la API)

Los verbos indican **qué operación realizar sobre un recurso**.

4.1 GET → Consultar información

Se usa para leer datos.

No modifica nada.

```
GET /productos
```

Respuesta (en JSON):

```
[
  {
    "id": 1,
    "nombre": "Laptop",
    "precio": 3500000
  },
  {
    "id": 2,
    "nombre": "Mouse",
    "precio": 80000
  }
]
```

4.2 POST → Crear información

Se usa para crear un nuevo recurso.

POST /productos

Datos enviados:

```
{
  "nombre": "Teclado",
  "precio": 150000
}
```

Respuesta:

```
{
  "id": 3,
  "nombre": "Teclado",
  "precio": 150000,
  "creado": true
}
```

4.3 PUT → Actualizar completamente

Reemplaza todo el recurso.

```
PUT /productos/3
{
  "nombre": "Teclado Mecánico",
  "precio": 220000
}
```

4.4 PATCH → Actualizar parcialmente

Solo modifica algunos campos.

```
PATCH /productos/3
{
  "precio": 200000
}
```

4.5 DELETE → Eliminar

DELETE /productos/3

Respuesta:

```
{
  "mensaje": "Producto eliminado"
}
```

5. ¿Qué es JSON?

JSON significa **JavaScript Object Notation**.

Es un formato de texto utilizado para **intercambiar datos entre sistemas**.

JSON no ejecuta código.

JSON no tiene lógica.

JSON solo transporta información estructurada.

Es como un “sobre digital” con datos.

6. Estructura Básica de JSON

Un JSON se construye con:

- { } → objetos
- [] → listas
- "clave": valor → atributos

Ejemplo:

```
{
  "nombre": "Ana",
  "edad": 28,
  "activo": true
}
```

7. Tipos de Datos en JSON

Tipo	Ejemplo
Texto	"Juan"
Número	100
Booleano	true
Objeto	{ }
Lista	[]
Nulo	null

8. JSON con Objetos Anidados

```
{
  "id": 10,
  "cliente": {
    "nombre": "Laura Gómez",
    "email": "laura@email.com"
  }
}
```

```
    },  
    "total": 500000  
}
```

Aquí hay un objeto dentro de otro.

9. JSON con Listas

```
{  
  "pedidoId": 501,  
  "productos": [  
    {  
      "nombre": "Monitor",  
      "precio": 900000  
    },  
    {  
      "nombre": "Cable HDMI",  
      "precio": 50000  
    }  
  ]  
}
```

Representa una colección de elementos.

10. Comunicación Completa entre Cliente y API

Solicitud

```
POST /usuarios  
{  
  "nombre": "Carlos Ruiz",  
  "email": "carlos@email.com",  
  "activo": true  
}
```

Respuesta del servidor

```
{  
  "id": 101,  
  "nombre": "Carlos Ruiz",  
  "email": "carlos@email.com",  
  "fechaCreacion": "2026-02-12"  
}
```

Así se comunican prácticamente todas las aplicaciones actuales.

11. Códigos de Respuesta HTTP

Código	Significado
200	Operación exitosa
201	Recurso creado
400	Solicitud incorrecta
401	No autorizado
404	No encontrado
500	Error del servidor

12. Buenas Prácticas al Diseñar APIs

- Usar sustantivos en las URLs.
- Usar correctamente los verbos HTTP.
- Responder siempre en JSON.
- Mantener estructura consistente.
- No mezclar lógica con los datos.
- Diseñar APIs pensando en otros desarrolladores.

13. Conclusión

Las APIs permiten la comunicación entre sistemas desacoplados.

Los verbos HTTP indican la acción que se quiere realizar.

JSON es el formato estándar que transporta los datos.

Comprender estos tres elementos es fundamental para trabajar con:

- Aplicaciones web modernas
- Microservicios
- Integraciones empresariales
- Arquitecturas distribuidas